Ejercicios de Expresiones Regulares con JavaScript

Estos ejercicios están diseñados para poner a prueba tu comprensión y habilidad con las expresiones regulares en JavaScript. Intenta resolverlos utilizando los conceptos y métodos explicados en el manual. Las soluciones se proporcionarán en un archivo separado.

Parte 1: Conceptos Básicos y Metacaracteres

1. **Validar un número de teléfono simple:** Crea una expresión regular que valide un número de teléfono de 9 dígitos, donde todos los dígitos son numéricos. La cadena debe contener *solo* el número de 9 dígitos.
   - `"123456789"` (válido)
   - `"12345678"` (inválido)
   - `"1234567890"` (inválido)
   - `"123A56789"` (inválido)
2. **Buscar palabras que empiecen con vocal:** Escribe una expresión regular que encuentre todas las palabras en una cadena que comiencen con una vocal (a, e, i, o, u), sin importar mayúsculas o minúsculas. Usa la bandera `g`.
   - Texto: `"El elefante es un animal enorme y elegante."`
   - Coincidencias esperadas: `["El", "elefante", "es", "un", "animal", "enorme", "y", "elegante"]`
3. **Contar ocurrencias de un carácter:** Utiliza una expresión regular para contar cuántas veces aparece la letra 'a' (mayúscula o minúscula) en una cadena.
   - Texto: `"Banana y Manzana son frutas sabrosas."`
   - Coincidencias esperadas: 6

Parte 2: Cuantificadores y Grupos

1. **Extraer etiquetas HTML:** Crea una expresión regular que extraiga el nombre de la etiqueta (por ejemplo, `div`, `p`, `span`) de cualquier etiqueta HTML de apertura o cierre. No debes capturar los corchetes `<` o `>`.
   - Texto: `"<div><p><span>Texto</span></p></div>"`
   - Coincidencias esperadas: `["div", "p", "span", "span", "p", "div"]`
2. **Validar formato de fecha (YYYY-MM-DD):** Escribe una expresión regular que valide si una cadena tiene el formato `YYYY-MM-DD`. Los años deben ser de 4 dígitos, y el mes y el día de 2 dígitos. No te preocupes por la validez real de la fecha (ej. 30 de febrero).

- `"2023-01-15"` (válido)
- `"1999-12-31"` (válido)
- `"2023-1-15"` (inválido)
- `"23-01-15"` (inválido)

3. **Reemplazar múltiples espacios:** Crea una expresión regular para reemplazar cualquier secuencia de uno o más espacios en blanco por un solo espacio.
   - Texto: `"Este texto tiene muchos espacios."`
   - Resultado esperado: `"Este texto tiene muchos espacios."`

Parte 3: Casos de Uso Avanzados

1. **Extraer nombres de usuario de Twitter:** Escribe una expresión regular que extraiga todos los nombres de usuario de Twitter (que comienzan con `@` y pueden contener letras, números y guiones bajos) de una cadena.
   - Texto: `"Hola @usuario1, ¿cómo estás? @otro_usuario y @tercerUsuario también están aquí."`
   - Coincidencias esperadas: `["usuario1", "otro_usuario", "tercerUsuario"]`
2. **Parsear parámetros de URL:** Crea una expresión regular para extraer pares clave-valor de los parámetros de consulta de una URL. Por ejemplo, de `?nombre=Juan&edad=30&ciudad=Madrid`, deberías obtener `["nombre=Juan", "edad=30", "ciudad=Madrid"]`.
   - Texto: `"https://ejemplo.com/pagina?nombre=Juan&edad=30&ciudad=Madrid"`
   - Coincidencias esperadas: `["nombre=Juan", "edad=30", "ciudad=Madrid"]`
3. **Encontrar palabras repetidas consecutivamente:** Escribe una expresión regular que encuentre palabras que se repiten consecutivamente en una cadena, como "hola hola" o "adiós adiós".
   - Texto: `"Esto es un ejemplo de texto con palabras repetidas repetidas como hola hola."`
   - Coincidencias esperadas: `["repetidas repetidas", "hola hola"]`

Parte 4: Métodos de `String` y `RegExp`

1. **Usar `matchAll` para extraer emails y dominios:** Utiliza `String.prototype.matchAll()` junto con una expresión regular para extraer tanto la dirección de correo electrónico completa como su dominio de una cadena.
   - Texto: `"Contacta a info@ejemplo.com o soporte@miempresa.net."`

- Coincidencias esperadas (estructura de ejemplo): `[[email: "info@ejemplo.com", dominio: "ejemplo.com"], [email: "soporte@miempresa.net", dominio: "miempresa.net"]]`

2. **Validación de código postal (5 dígitos):** Utiliza `RegExp.prototype.test()` para verificar si una cadena es un código postal válido de 5 dígitos numéricos.
   - `"28001"` (válido)
   - `"1234"` (inválido)
   - `"28001-1234"` (inválido)
   - `"ABCDE"` (inválido)
3. **Reemplazar fechas a formato DD/MM/YYYY:** Usa `String.prototype.replace()` con grupos de captura para transformar fechas del formato `YYYY-MM-DD` a `DD/MM/YYYY`.
   - Texto: `"La fecha de inicio es 2024-03-10 y la final es 2025-12-25."`
   - Resultado esperado: `"La fecha de inicio es 10/03/2024 y la final es 25/12/2025."`